Übung 5

Document Object Model (DOM)

Einführung in Webengineering · WS 2026/27

In sieben Aufgaben untersuchen Sie den DOM-Baum und verändern Webseiten mit JavaScript. Begründen Sie Ihre Vermutungen und prüfen Sie diese anschließend im Browser.

01 DOM-Struktur und Ladezeitpunkt verstehen

Betrachten Sie diesen HTML-Ausschnitt:

```
<div id="karte">Hallo <strong>DOM</strong><!-- Hinweis --></div>
```

a) Zeichnen Sie den Teilbaum ab dem `div`, einschließlich Text- und Kommentarknoten. Welche Knoten sind Elemente? Bestimmen Sie für das `div` die Werte von `children.length` und `childNodes.length`. Übernehmen Sie den Ausschnitt dazu ohne zusätzliche Zeilenumbrüche.

b) Erläutern Sie den Unterschied zwischen `window`, `document` und `document.documentElement`. Welches Objekt steht für den Dokumentknoten, welches für das `html`-Element?

c) Ein klassisches Skript im `head` wird vor dem obigen `div` ausgeführt:

```
document.getElementById("karte").textContent = "Hallo DOM";
```

Warum schlägt der Zugriff fehl? Korrigieren Sie die Position des Skripts und erklären Sie, weshalb der Zugriff danach funktioniert.

02 Elemente auswählen und Attribute ändern

Verwenden Sie diesen Ausschnitt:

```
<ul id="themen">
  <li class="thema">HTML</li>
  <li class="thema aktuell" data-stufe="2">DOM</li>
  <li class="thema">Events</li>
</ul>
<a id="info" href="details.html">Weitere Informationen</a>
```

a) Wählen Sie die Liste mit `getElementById` aus. Vergleichen Sie darauf die Ergebnisse von `querySelector(".thema")`, `querySelectorAll(".thema")` und `getElementsByClassName("thema")`. Geben Sie mit `Array.from(...).forEach(...)` die Texte der letzten Ergebnismenge aus. Warum ist die Umwandlung hier hilfreich?

b) Wählen Sie `.aktuell` aus und ändern Sie über `nextElementSibling` den Text des nächsten Listeneintrags in „Ereignisse“. Lesen Sie `data-stufe` über `dataset` aus, erhöhen Sie den Wert um 1 und speichern Sie ihn zurück. Achten Sie auf den Datentyp.

c) Vergleichen Sie beim Link `getAttribute("href")` mit der Eigenschaft `href`. Erklären Sie den Unterschied. Setzen Sie das Attribut anschließend mit `setAttribute` auf `hilfe.html`.

Inhalte verändern und auf Eingaben reagieren

03 Inhalte lesen und DOM-Knoten bearbeiten

Verwenden Sie diese beiden Bereiche:

```
<p id="text">Hallo <strong>DOM</strong><span hidden>!</span></p>
<ul id="liste">
  <li>HTML</li>
  <li>CSS</li>
</ul>
```

a) Lesen Sie beim Absatz `innerHTML`, `outerHTML`, `textContent` und `innerText` aus. Erklären Sie die Unterschiede anhand der Tags und des ausgeblendeten Ausrufezeichens.

b) Weisen Sie dem Absatz zuerst über `textContent`, dann über `innerHTML` die Zeichenkette `"<em>DOM verstehen</em>"` zu. Laden Sie vor jedem Versuch die Seite neu. Was wird jeweils angezeigt? Was würde bei einer Zuweisung an `outerHTML` zusätzlich ersetzt?

c) Ergänzen Sie die Liste ausschließlich mit DOM-Methoden: Erzeugen Sie einen Eintrag „JavaScript“ mit `createElement` und `createTextNode` und hängen Sie ihn mit `appendChild` an. Verschieben Sie diesen bestehenden Eintrag mit `insertBefore` an den Anfang und entfernen Sie „CSS“ mit `removeChild`. Wird beim Verschieben eine Kopie erzeugt?

04 Eingaben verarbeiten und Listener registrieren

Legen Sie eine eigene Seite mit diesem Inhalt an:

```
<label for="nachricht">Nachricht</label>
<textarea id="nachricht"></textarea>
<p id="vorschau"></p>
<button id="test" type="button">Listener testen</button>
```

a) Binden Sie eine externe JavaScript-Datei am Ende von `body` ein. Registrieren Sie mit `addEventListener` einen Listener für `input`. Übertragen Sie darin den vollständigen aktuellen Eingabetext aus `value` mit `textContent` in die Vorschau. Verzichten Sie auf Event-Attribute im HTML.

b) Geben Sie im Listener zusätzlich `event.data` aus. Vergleichen Sie den Wert beim Tippen und Löschen mit `value`. Weshalb eignet sich `event.data` nicht als vollständiger Vorschautext?

c) Registrieren Sie am Button zwei verschiedene `click`-Listener, die „A“ bzw. „B“ ausgeben. Vergleichen Sie das Ergebnis mit zwei aufeinanderfolgenden Zuweisungen an `onclick`. Prüfen Sie beide Varianten getrennt nach einem Neuladen der Seite.

Den Weg eines Ereignisses nachvollziehen

05 Capturing, Target und Bubbling unterscheiden

Verwenden Sie dieses HTML und führen Sie das Skript danach aus:

```
<div id="aussen">
  <div id="innen">
    <button id="knopf" type="button">Klicken</button>
  </div>
</div>
```

```
const aussen = document.getElementById("aussen");
const innen = document.getElementById("innen");
const knopf = document.getElementById("knopf");
function log(event) {
  console.log(this.id, event.target.id, event.eventPhase);
}
aussen.addEventListener("click", log, true);
innen.addEventListener("click", log, true);
knopf.addEventListener("click", log);
innen.addEventListener("click", log);
aussen.addEventListener("click", log);
```

a) Notieren Sie die fünf erwarteten Ausgaben für einen Klick auf den Button. Prüfen Sie Ihre Vorhersage anschließend im Browser. Ordnen Sie die Zahlen von `event.eventPhase` den Phasen Capturing, Target und Bubbling zu.

b) Warum bleibt `event.target.id` gleich, während sich `this.id` ändert? Erklären Sie die Wirkung von `true` anhand der beiden Listener am äußeren `div`.

c) Entfernen Sie die beiden Registrierungen mit `true`. Ersetzen Sie die Registrierung von `log` am inneren `div` durch diese beiden Listener; die Listener am Button und am äußeren `div` bleiben bestehen:

```
innen.addEventListener("click", function (event) {
  console.log("innen A");
  event.stopPropagation();
});
innen.addEventListener("click", function () {
  console.log("innen B");
});
```

Welche Ausgaben erscheinen jetzt beim Button-Klick? Ersetzen Sie danach `stopPropagation()` durch `stopImmediatePropagation()` und vergleichen Sie. Laden Sie die Seite vor jedem Versuch neu, damit keine alten Listener aktiv bleiben.

Browseraktionen und Darstellung steuern

06 Standardaktionen und eigene Events

Verwenden Sie eine eigene Seite mit diesem Inhalt:

```
<div id="bereich">
  <a id="sprung" href="#ziel">Zum Ziel</a>
</div>
<h2 id="ziel">Ziel</h2>
<p id="status"></p>
```

a) Registrieren Sie am Link einen `click`-Listener, der mit `preventDefault()` den Sprung verhindert. Ein weiterer `click`-Listener am umgebenden `div` soll „Klick im Bereich“ ausgeben. Prüfen Sie die Adresszeile und die Konsole. Warum wird der zweite Listener trotzdem ausgeführt?

b) Erzeugen Sie beim Link-Klick außerdem ein `CustomEvent` namens `thema-gewaehlt` mit `detail: { thema: "DOM", kapitel: 5 }`. Lösen Sie es mit `dispatchEvent` am `div` aus. Ein dort registrierter Listener für dieses eigene Event soll aus `event.detail` den Status „Kapitel 5: DOM“ zusammensetzen. Erklären Sie die Rollen von Event-Typ, `detail` und `dispatchEvent`.

07 CSS-Klassen und Sichtbarkeit steuern

Nutzen Sie dieses HTML und die folgenden CSS-Klassen:

```
<div id="box" class="karte">DOM und CSS</div>
<p>Dieser Absatz folgt auf die Box.</p>
<button id="markieren" type="button">Markierung umschalten</button>
```

```
.markiert { color: navy; background-color: gold; }
.ausgeblendet { display: none; }
.unsichtbar { visibility: hidden; }
```

a) Schalten Sie bei jedem Button-Klick mit `classList.toggle` die Klasse `markiert` an der Box um. Die vorhandene Klasse `karte` soll erhalten bleiben. Prüfen Sie, dass ein zweiter Klick die ursprüngliche Darstellung wiederherstellt.

b) Erzeugen Sie dieselben Farben versuchsweise direkt über `style.color` und `style.backgroundColor`. Welchen Vorteil hat die Klassenlösung für die Trennung von JavaScript und CSS? Entfernen Sie die Inline-Stile nach dem Vergleich wieder.

c) Testen Sie die Klassen `ausgeblendet` und `unsichtbar` einzeln an der Box. Beobachten Sie jeweils die Position des folgenden Absatzes. Prüfen Sie auch mit `getElementById`, ob die Box noch im DOM vorhanden ist. Erklären Sie den Unterschied zwischen beiden Varianten.